```
Failed examples:

rspec ./spec/unit/app/api_spec.rb:24 # ExpenseTracker::API POST /expenses ↩
when the expense is successfully recorded returns the expense id
rspec ./spec/unit/app/api_spec.rb:31 # ExpenseTracker::API POST /expenses ↩
when the expense is successfully recorded responds with a 200 (OK)
rspec ./spec/unit/app/api_spec.rb:53 # ExpenseTracker::API POST /expenses ↩
when the expense fails validation responds with a 422 (Unprocessable entity)
rspec ./spec/unit/app/api_spec.rb:46 # ExpenseTracker::API POST /expenses ↩
when the expense fails validation returns an error message

Randomized with seed 40684
```

Notice the complaint: the ExpenseTracker::Ledger class does not implement the instance method: record. The specs are failing because the real Ledger class doesn't act enough like the fake one.

You've just encountered a feature of RSpec called *verifying doubles*. They help prevent *fragile mocks*, a problem where specs pass when they should be failing.

Recall that test doubles mimic the interface of a real object. When the real object's method names or parameters change, a traditional test double will still respond to the old methods. It's easy to forget to update your specs when this happens.

RSpec's verifying doubles actually inspect the real object they're standing in for and fail the test if the method signatures don't match. You'll learn more about them in *Verifying Doubles*, on page 243.

Let's follow the guidance of the error message and add an empty record method to Ledger:

05-unit-specs/09/expense_tracker/app/ledger.rb

```
def record
end
```

Now, rerun the spec:

```
$ bundle exec rspec spec/unit/app/api_spec.rb
« truncated »

Failures:

  1) ExpenseTracker::API POST /expenses when the expense is successfully ↩
  recorded responds with a 200 (OK)
     Failure/Error:
       allow(ledger).to receive(:record)
         .with(expense)
         .and_return(RecordResult.new(true, 417, nil))

       Wrong number of arguments. Expected 0, got 1.
     # ./spec/unit/app/api_spec.rb:19:in `block (4 levels) in ↩
     <module:ExpenseTracker>'
```